



# 04. The Abstraction: The Process

Until Now

Process

Time-Sharing

Process API

Process Creation

1. Program Code를 Memory에 Load
2. Program의 Run-Time Stack이 할당
3. Program의 Heap 영역이 생성
4. 다른 initialization tasks
5. **Start the program**

Process States

Ready

Running

Blocked

I/O Operation Flow

- 1 **Application이 System Call 호출**
- 2 **OS의 File System & Driver가 작동**
- 3 **DMA가 Kernel Memory로 데이터 복사** ★
- 4 **Kernel Memory → User Memory로 복사**
- 5 **System Call 완료, User Mode로 복귀**

PCB(Process Control Block)

Process List

## Until Now

- 지난 글: OS의 핵심인 3대 요소는 **Virtualization, Concurrency, Persistence**이다.
- Virtualization의 두 가지 유형이 존재.
- 1. **Process Virtualization**  
**Process context switch**를 통한 **time-sharing**이 기반  
우선 여기에 대해서 먼저 알아볼 것

## 2. Memory Virtualization

Virtual Memory Address Space가 기반

# Process

## | Running instance of a program

- program을 바라보는 2가지 관점

### 1. **Passive Entity**

disk에 executable file 형태로 저장되어 있는 'List of Instructions' File에 불과하다.

### 2. **Running Entity**

disk로부터 main memory로 올라가면, CPU는 Program Counter를 이용해 program을 실행한다.



Program이 Disk에서는 Executable File로 존재  
Main Memory로 Load되면 Process라고 함.

- process의 구성요소
  - Memory (address space)
    - Instructions
    - Data Section
  - Registers
    - Program Counter
    - Stack Pointer

## Time-Sharing

- 하나의 Process가 수행되고, 중간에 멈춘 다음, 이어서 다른 Process가 수행되고,.. 이런 과정이 매우 짧은 시간을 텀으로 두어 반복해 '마치 각 프로세스가 자신의 CPU를 두고 동시에 수행되는 것처럼 보이게 하는 현상'을 일으킴
- Context Switch 시의 Overhead가 존재하기 때문에 Performance(Potential Cost)를 고려해야 함

## Process API

- **Create**: Create a new process to run a program
  - **Destroy**: Halt a runaway process
  - **Wait**: Wait for a process to stop running
  - **Miscellaneous Control**: Some kind of method to suspend a process and then resume it
  - **Status**: Get some status info about a process
- 

## Process Creation

### 1. Program Code를 Memory에 Load

- 프로그램을 메인 메모리에 로드 - 원래는 disk에서 executable한 format으로 있음
- Loading을 lazy하게 수행
  - Program Execution에 필요한 **Code, Data Section**만 우선적으로 로딩

### 2. Program의 Run-Time Stack이 할당

- Local variables(지역변수)
- Function parameters(함수 매개변수)
- Return address(끝나면 어디로 돌아갈지 기억하는 주소)
- 가장 먼저 스택에 쌓이는 건 main 함수의 parameters → argc and the argv array of main() function

### 3. Program의 Heap 영역이 생성

- Program 수행 중 Dynamic한 Explicit Allocate-Request
- malloc() & free()

### 4. 다른 initialization tasks

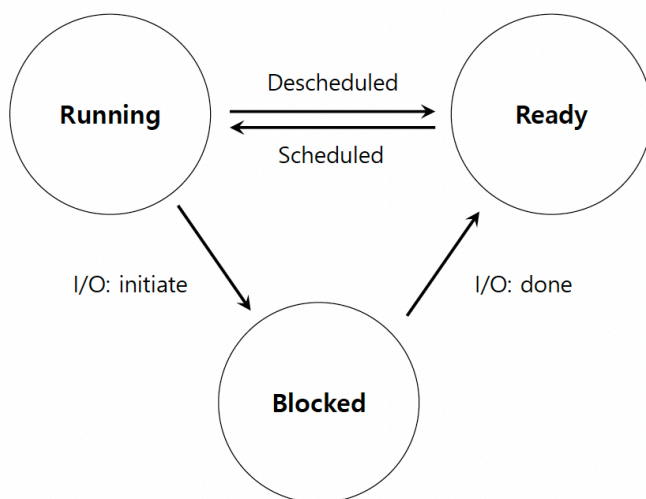
- input/output (I/O) setup

- STDIN: 0번
- STDOUT: 1번
- STDERR: 2번

## 5. Start the program

- running at the entry point: `main()`
  - PC가 main function을 가리키며 수행 시작

## Process States



### Ready

- running할 준비는 되어 있는데, 아직 OS로부터 선택받지 않아 schedule 되어 있지 않은 상태
- main memory에는 이미 적재되어 있는 상태
- queue에 보관되어 있음

### Running

- process가 CPU control을 지닌채 processor에서 실행되는 상태

## Blocked

- I/O나 Event 대응 등의 연산을 수행하는 상태
- I/O operation이 완료 등 block상태가 해제되면 ready state으로 전이

| Time | Process <sub>0</sub> | Process <sub>1</sub> | Notes                         |
|------|----------------------|----------------------|-------------------------------|
| 1    | Running              | Ready                |                               |
| 2    | Running              | Ready                |                               |
| 3    | Running              | Ready                |                               |
| 4    | Running              | Ready                | Process <sub>0</sub> now done |
| 5    | –                    | Running              |                               |
| 6    | –                    | Running              |                               |
| 7    | –                    | Running              |                               |
| 8    | –                    | Running              | Process <sub>1</sub> now done |

Figure 4.3: Tracing Process State: CPU Only

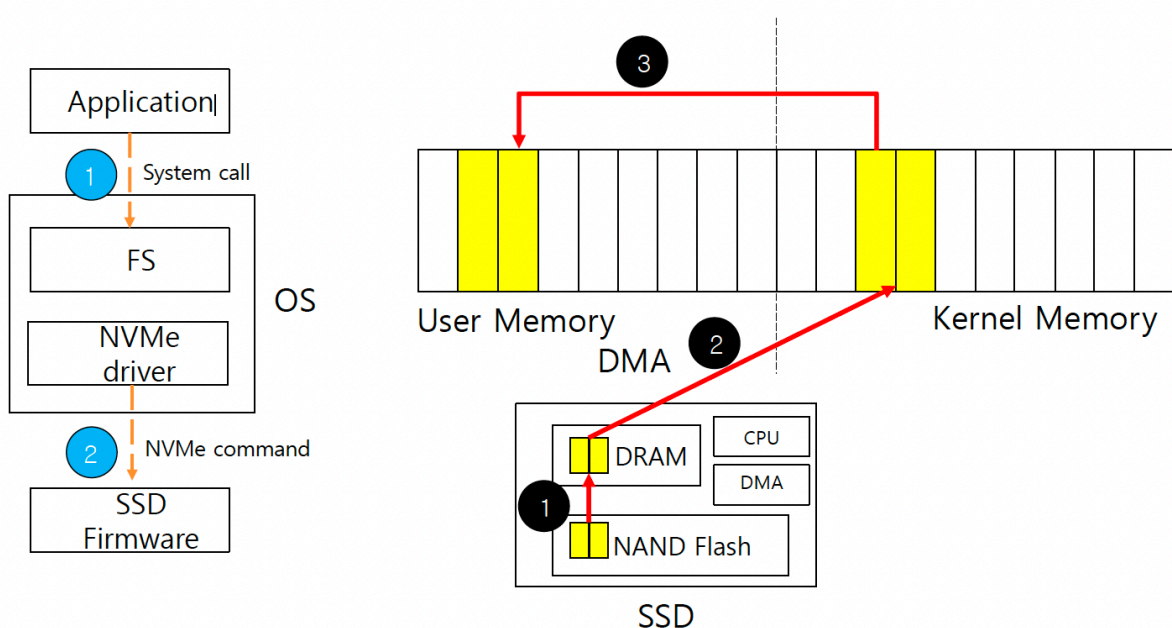
| Time | Process <sub>0</sub> | Process <sub>1</sub> | Notes                                                         |
|------|----------------------|----------------------|---------------------------------------------------------------|
| 1    | Running              | Ready                |                                                               |
| 2    | Running              | Ready                |                                                               |
| 3    | Running              | Ready                | Process <sub>0</sub> initiates I/O                            |
| 4    | Blocked              | Running              | Process <sub>0</sub> is blocked, so Process <sub>1</sub> runs |
| 5    | Blocked              | Running              |                                                               |
| 6    | Blocked              | Running              |                                                               |
| 7    | Ready                | Running              | I/O done                                                      |
| 8    | Ready                | Running              | Process <sub>1</sub> now done                                 |
| 9    | Running              | –                    |                                                               |
| 10   | Running              | –                    | Process <sub>0</sub> now done                                 |

Figure 4.4: Tracing Process State: CPU and I/O



'I/O Operation 완료'와 같은 이유로 Block이 해제되면,  
바로 Running State가 되는 것이 아니라, Ready State가 된다.

## I/O Operation Flow



## 1 Application이 System Call 호출

```
read(fd, buffer, size); // 유저가 파일 읽기 요청
```

- User mode에서 실행 중인 애플리케이션이 데이터를 읽고 싶어함
- System call을 통해 kernel mode로 전환됨 (trap 발생)

## 2 OS의 File System & Driver가 작동

- **FS(File System)**: 어떤 파일의 어느 위치를 읽어야 하는지 파악
- **NVMe Driver**: SSD에 명령을 내림 (NVMe command 전송)
- **SSD Firmware**: 실제로 NAND Flash에서 데이터를 읽어서 SSD 내부 DRAM 버퍼로 가져옴  
여기서 말하는 DRAM은 SSD 내부의 DRAM (버퍼 캐시)

## 3 DMA가 Kernel Memory로 데이터 복사 ★

- **DMA(Direct Memory Access)** 컨트롤러가 CPU의 개입 없이 데이터를 옮김
- SSD의 DRAM → **Kernel Memory**의 버퍼로 복사됨
- 이 과정에서 CPU는 다른 일을 할 수 있어 (비동기적!)

## 4 Kernel Memory → User Memory로 복사

- 이제 kernel buffer에 있는 데이터를
- **User Memory** (애플리케이션의 buffer)로 복사함
- 이걸 보통 CPU가 직접 수행 (memcpy 같은 함수로)

## 5 System Call 완료, User Mode로 복귀

- read() 함수가 리턴되면서 애플리케이션이 데이터를 사용 가능

## PCB(Process Control Block)

- process의 중요 정보 저장
- PID(Process ID), PC(Program Counter), Registers(수행하면서 사용한 데이터 값들), Memory 제한, Open File 정보 등

```

struct proc {                // xv6의 PCB 구조 ★★★
    char *mem;                // Process Memory의 시작점
    uint sz;                  // Process Memory의 사이즈
    char *kstack;             // 해당 Process에 대한 Kernel Stack의 Bottom
    enum proc_state state;    // Process State (enum Type으로 나타냄)
    int pid;                  // Process ID
    struct proc *parent;      // Parent Process
    void *chan;
    int killed;               // Killed인 경우 양수를 대입한다.
    struct file *ofile[NOFILE]; // Open Files
    struct inode *cwd;         // Current Working Directory (현위치)
    struct context context;    // Process 수행을 위해 Switch하는 부분 (Registers)
    struct trapframe *tf;     // 현재 Interrupt에 대한 Trap Frame
};

enum proc_state { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

struct context {             // Register Context (Registers)
    int eip;                  // Index Pointer Register
    int esp;                  // Stack Pointer Register
    int ebx;                  // Base Register
    int ecx;                  // Counter Register
    int edx;                  // Data Register
    int esi;                  // Source Index Register
    int edi;                  // Destination Index Register
    int ebp;                  // Stack Base Pointer Register
};

```

## Process List

- 메모리에 상주하고 있는, 이미 Creation이 이루어진 Process들을 연결시켜서 유지/관리하는 자료구조
- Queue의 구조를 가지며, PCB를 Node로 함
  - Ready Queue, Blocked Queue, Current running processes

